Design LLM Inference API Service

題目描述（Problem Statement）

假設你可以引入一個函式庫，其中定義了一個 batched_sample 函式，用於透過大型語言模型（LLM）生成文字。此函式必須在雲端供應商的昂貴 GPU instance 上執行。

batched_sample 接受一個「批次（batch）」的字串作為輸入，因為執行任何輸入都有一個無法避免的固定延遲（irreducible fixed latency），因此無論 batch 中有多少筆資料，函式的回傳時間始終約為 100ms。它支援 1-100 筆字串的 batch 大小，且每台 GPU instance 同一時間只能處理一個 batch。

函式簽章如下：

```
def batched_sample(batched_inputs: List[str]) -> List[str]:
"""
給定一組字串（prompts），為每個字串生成一個 completion，
並以 list 的形式回傳所有 completions。

支援 1-100 筆輸入字串；任何 batch 大小的延遲皆約為 100ms。
若 batch 大小超過 100 則會拋出錯誤。

此函式只能在昂貴的 GPU server 上執行，
且在執行期間會完全佔用該 server 上的 GPU。

"""
# Example usage:
batched_sample([
    "E equals ",
    "when there's a will, "
])
# Returns
[
```

buildmoat.org

```
    "MC squared",
    "there's a way"
]
```

你的任務是設計一個 HTTP API，將上述函式暴露給外部使用者，讓他們能夠對大型語言模型進行 sampling。使用者期望能夠以單一請求的形式呼叫 API：

```
curl llm-api.buildmoat.com/sample -d "E equals "
# 回傳：
# MC squared
```

核心挑戰在於：使用者發送的是單一請求，但底層函式需要 batch 輸入。系統需要在兩者之間架起橋樑，同時兼顧延遲、吞吐量與成本。

功能性需求（Functional Requirements）

1. 使用者應該能夠透過 HTTP 請求提交單一 prompt，並取得 LLM 的 completion 回應。
2. 系統應該能夠將多個使用者的獨立請求聚合為 batch（批次），以最大化 GPU 使用效率。
3. 系統應該能夠管理一個 GPU instance pool，根據流量動態調度資源。

非功能性需求（Non-Functional Requirements）

1. **低延遲（Low latency）**：使用者從發送請求到收到回應，端到端延遲應控制在 200ms 以內。
2. **高吞吐量（High throughput）**：在流量尖峰期間，系統應該能夠透過高效的 batching 策略處理大量並發請求，不丟失任何請求。
3. **成本效率（Cost efficiency）**：GPU instance 價格昂貴，系統應最大化每個 batch 的填充率（batch utilization），並在低流量時自動縮減 GPU 數量。
4. **正確性（Correctness）**：每個使用者的請求必須取得對應其 prompt 的正確 completion，不得出現結果匹配錯誤。

API Design

buildmoat.org

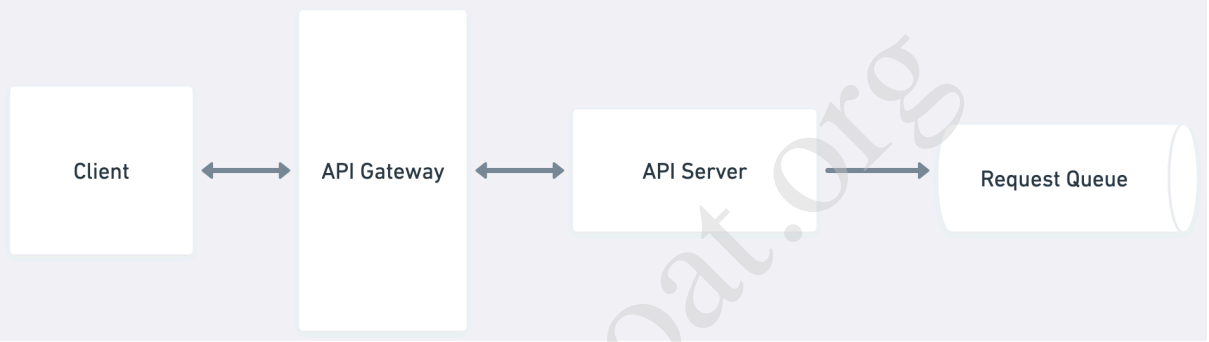
```
POST llm-api.buildmoat.com/sample

Body: <raw text prompt>

Response: <raw text completion>
```

High-Level Design

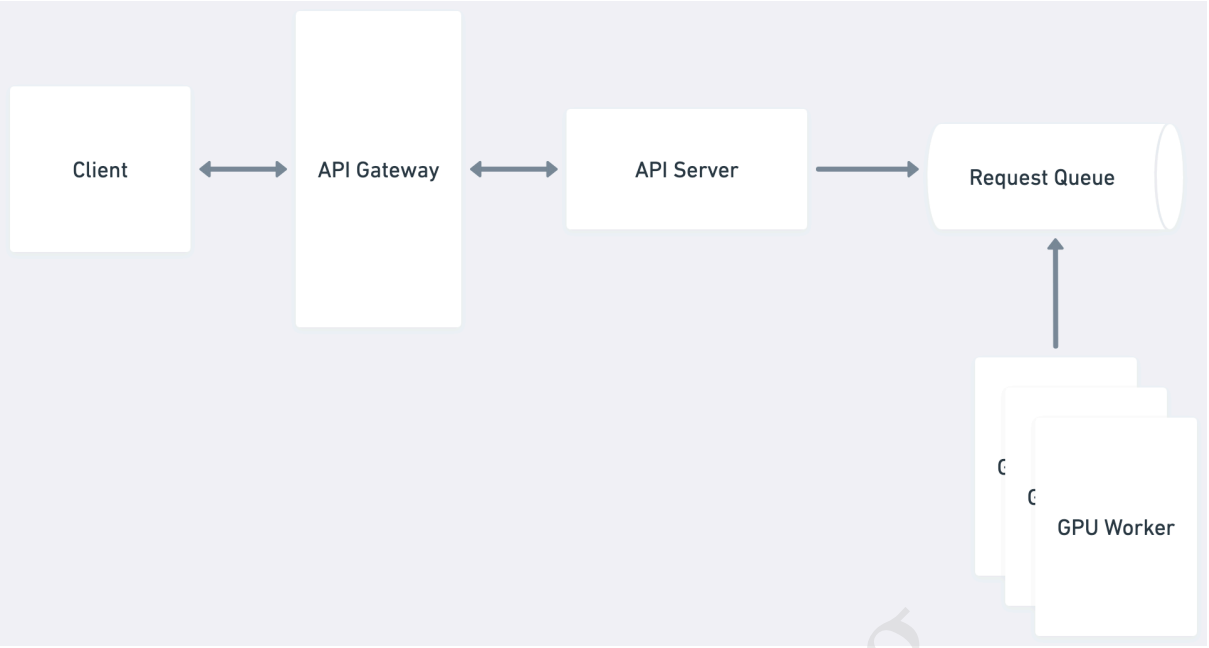
1. 使用者提交 prompt 並進入等待



1. 使用者透過 `POST /sample` 將 prompt 送至 **API Server**。
2. API Server 為該請求產生一個唯一的 `request_id`，並將 `(request_id, prompt)` 推入 **Request Queue**。

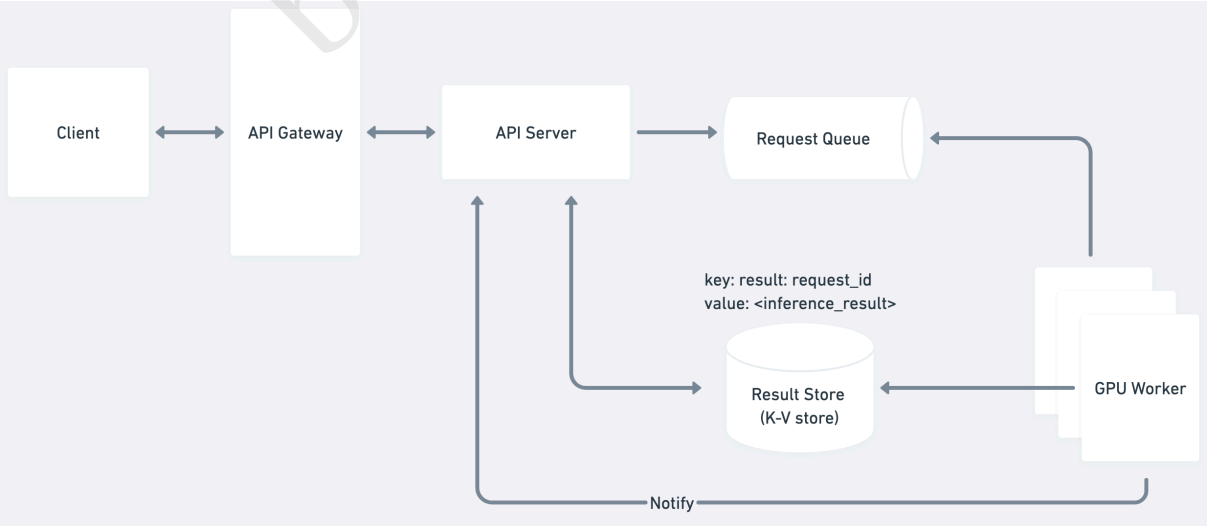
2. GPU Worker 拉取批次並執行推論

buildmoat.org



1. **GPU Worker** 持續從 Request Queue 中拉取請求，使用 **dual-trigger batching**（雙觸發批次策略）來決定何時形成一個 batch：
   - **Size trigger**：當收集到 100 筆請求時，立即觸發執行。
   - **Time trigger**：當 batch 中第一筆請求等待超過 `max_wait`（例如 50ms）時，無論 batch 大小為何，立即觸發執行。
2. GPU Worker 將 batch 中所有 prompt 送入 `batched_sample(prompts)` 執行推論（耗時 ~100ms）。

3. 結果回傳給使用者



buildnoat.org

1. 在第一部分的最後，API Server 隨後 subscribe Result Store 中 `result:{request_id}` 對應的通知，等待結果寫入。
2. GPU Worker 取得結果後，對 batch 中每一筆請求：
   a. 將 completion 寫入 **Result Store**（KV store），key 為 `result:{request_id}`，並設定 TTL 自動過期。
   b. 發佈通知，告知 API Server 結果已就緒。
3. API Server 收到通知後，從 Result Store 讀取 completion 並回傳給使用者，關閉 HTTP 連線。

> 為什麼 GPU Worker 不直接將結果推送給 API Server？

> 使用者的請求經過 Load Balancer 後，被分配到某一台 API Server，由該 API Server 持有 HTTP 連線。但 GPU Worker 從 queue 中拉取請求時，無法得知是哪一台 API Server 正在等待這筆結果。
> 若要實現直接推送，GPU Worker 需要知道每個 request_id 對應的 API Server 位址，這代表需要額外維護一個 request_id → API Server 的 mapping，增加了元件間的耦合與系統複雜度。
> 此外，若持有連線的 API Server 在等待期間 crash，GPU Worker 的推送會失敗，需要額外的錯誤處理邏輯。
> 透過 Result Store 作為中介層，GPU Worker 只需寫入共享的 Result store（職責單一），API Server 透過通知機制自行取得結果（解耦），兩者之間不需要知道對方的存在。

buildmoat.org

深入探討（Deep Dives）

1. 我們要如何設定 batching 策略，以平衡延遲與吞吐量？

Batching 策略的核心取捨在於：**等待越久，batch 越滿，GPU 利用率越高；但使用者等待時間也越長**。

前述的 dual-trigger batching 在不同流量下的表現如下：

場景	批次等待	GPU 處理	總延遲（不含網路）
高流量（batch 迅速填滿）	~0-10ms	100ms	~100-110ms
中流量	~20-40ms	100ms	~120-140ms
低流量（觸發 time trigger）	50ms	100ms	~150ms

`max_wait` 的選擇：

- 過短（如 5ms）：中低流量下 batch 會很小（如 1-5 筆），GPU 利用率低，成本浪費嚴重。
- 過長（如 500ms）：使用者延遲不可接受。
- 建議起始值為 **50ms**，根據 P99 latency 與 GPU utilization metrics 動態調整。

進階優化 — **Adaptive batching**：根據當前 queue depth 動態調整 `max_wait`：

- Queue depth > 100：`max_wait = 0`（不等待，直接拉滿 100 筆，追求最大吞吐）。
- Queue depth < 10：`max_wait = 50ms`（給予更多等待時間以增加 batch 大小，提升 GPU 利用率）。

2. Request Queue 和 Result Store 的技術選型

Request Queue — Redis Streams：

- 選擇 Redis Streams 而非 Redis List 或外部 message queue（如 Kafka、SQS），基於以下考量：
  - **Consumer group 機制**：多個 GPU Worker 可安全地並行消費，每條訊息只會被分配給一個 worker，不會重複消費。
  - **內建 at-least-once delivery**：worker 讀取訊息後，訊息進入 Pending Entries List（PEL），直到 worker 送出 ACK 才算處理完成。若 worker crash 未 ACK，其他 worker 可在 idle timeout 後 claim 這些訊息並重新處理——這等同於 SQS 的 visibility timeout，但不需要引入外部服務。
  - **無拉取數量限制**：一次可拉取任意數量的訊息（透過 COUNT 參數），完美匹配 `batched_sample` 的 1-100 筆需求。SQS 每次最多只能拉取 10 條，需要多次呼叫才能湊滿一個 batch。
  - **Sub-millisecond 延遲**：Redis 為 in-memory store，遠快於 SQS 的 5-20ms per call 或 Kafka 的磁碟寫入延遲。

> **為什麼不選 SQS？**
>
> - SQS 作為全託管的 message queue，原生支援 visibility timeout、Dead Letter Queue、跨 AZ 持久化，在可靠性和維運成本上有明顯優勢。然而在此系統中，它有兩個關鍵限制：
> - **拉取數量上限為 10 條**：`batched_sample` 支援 100 筆，但 SQS 的 receive 每次最多只能拉取 10 條訊息。GPU Worker 需要多次呼叫（可並行）才能湊

buildmoat.org

滿一個 batch，這增加了實作複雜度。

  - **延遲顯著較高**：SQS 每次呼叫約 5-20ms。即使並行拉取 10 次，總延遲仍約 20-50ms，接近甚至超過 `batched_sample` 本身的 100ms 處理時間。對於端到端 SLA < 200ms 的即時 API，這個額外延遲是難以接受的。

- **選擇依據**：若系統是面向即時互動的 API（如本題），延遲為第一優先，選 Redis Streams。若系統是面向離線或批量任務（如 Batch API），可靠性比延遲重要，SQS 會是更好的選擇。

Result Store — Redis KV：

- 選擇 Redis KV 而非 message queue 作為 Result Store，因為 API Server 需要的是**按 `request_id` 精確查詢**某一筆結果，這是 key-value lookup 的存取模式，不適合 Kafka/SQS 等 log-based 或 queue-based 系統。

3. API Server 如何得知結果已就緒？——通知機制的選型

GPU Worker 將結果寫入 Result Store 後，需要一個機制通知 API Server。以下是三種可行方案：

方案 A：Polling

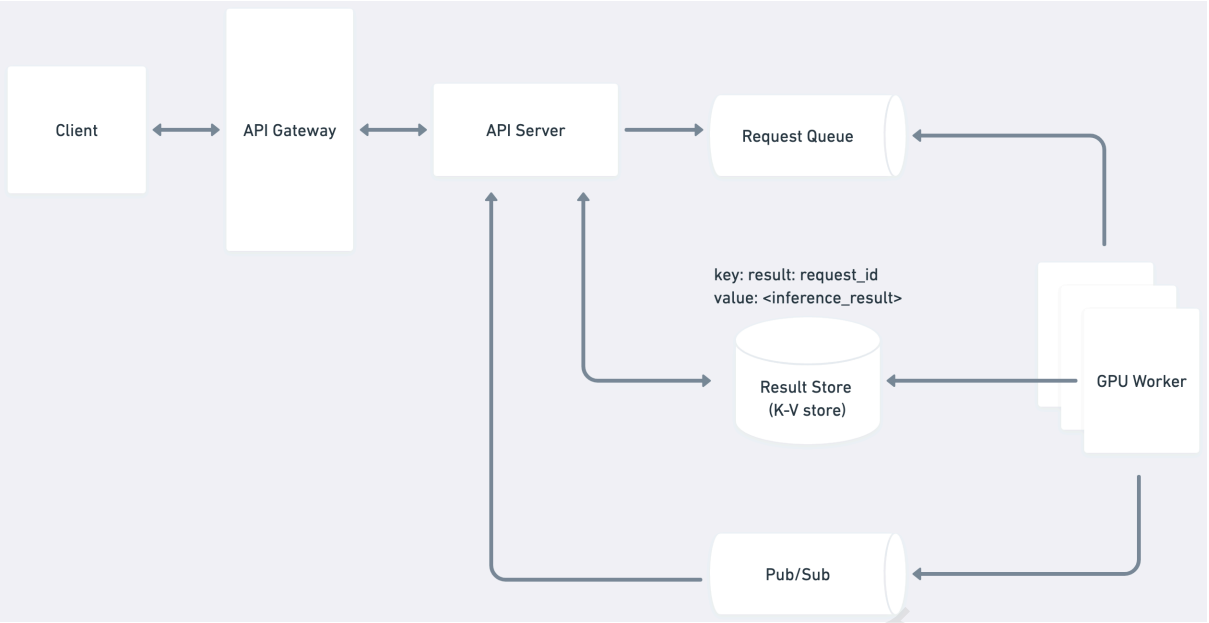
- API Server 定期輪詢 KV store，檢查 `result:{request_id}` 是否已存在。
- **優點**：實作極度簡單，不依賴任何特殊機制。
- **缺點**：浪費 Redis 讀取次數（1,000 並發 × 每 50ms 一次 = 20,000 GET/sec，多數為無效查詢），且延遲增加最多一個 poll interval。
- **適合場景**：Prototype 或極小規模部署。

方案 B：Blocking wait on Redis List

- GPU Worker 將結果寫入一個以 `result:{request_id}` 為 key 的 Redis List。
- API Server 對該 list 執行阻塞式讀取：若 list 為空，Redis server 端會 hold 住連線，直到有值寫入或 timeout。
- **優點**：實作最簡單，語意清晰，不存在 race condition（無論先寫還是先等，都能正確運作）。
- **缺點**：每個 in-flight 請求佔用一條 Redis 連線。1,000 並發 × 10 台 API Server = 10,000 條連線，已達 Redis 預設上限。
- **適合場景**：中小規模，並發量較低時的首選方案。

方案 C：Redis Pub/Sub + KV

buildmoat.org



- GPU Worker 完成後同時執行兩步：將結果寫入 KV，並透過 Pub/Sub 對 channel `result:{request_id}` 發佈通知。
- API Server 端每台僅維持 **1 條** subscribe 連線。對每個 in-flight 請求，透過這條共用連線 subscribe 對應的 channel。當任一 channel 收到訊息，Redis 會透過這條連線推送回來。
- **優點**：連線數從「每請求一條」降為「每台 API Server 一條」，解決了方案 A 的擴展性瓶頸。
- **缺點**：Pub/Sub 是 fire-and-forget，若 publish 發生在 subscribe 之前，訊息會遺失。因此
  - worker **先寫 KV** `SET result:{request_id}`
  - 再 `PUBLISH notify ...`
  - API server subscribe 後先 `GET result:{id}`（或 timeout 前再 GET）
- **適合場景**：高併發環境，連線數為瓶頸時的推薦方案。

建議演進路徑： 起步用方案 B（簡單可靠），當並發量高到 Redis 連線成為瓶頸時，遷移到方案 C。

4. 我們要如何處理流量尖峰，避免遺失任何使用者請求？

在流量突然激增時，GPU 處理速度可能跟不上請求進入的速度，導致 queue 不斷增長、延遲飆升。

第一層：Request Queue 作為緩衝

buildnoat.org

- Request Queue 天然具備緩衝功能。即使 GPU Worker 暫時無法消化所有請求，請求仍會安全地存放在 queue 中。
- 短暫的流量尖峰可完全被 queue 吸收，待 GPU Worker 消化後自然恢復。

第二層：GPU Auto-Scaling

- 引入 Auto-Scaler 服務，監控 queue depth 並動態調整 GPU Worker 數量。
  - Queue depth 持續超過 high watermark（例如 500）→ scale up。
  - Queue depth 持續低於 low watermark（例如 50）→ scale down，並設定 cooldown period 避免頻繁啟停。
- GPU instance 的 cold start 需要數十秒至數分鐘，因此可維持一個 **warm pool**（預熱池）：一組已啟動且完成模型載入的 GPU instance，需要時可立即投入使用。
- 可結合 **predictive scaling**，根據歷史流量模式（例如每日尖峰時段）提前啟動 GPU instance，緩解 cold start 問題。
- 無論流量多低，至少維持 **1 台** active GPU instance，確保 baseline availability。

第三層：Backpressure 與 Rate Limiting

- 當 queue depth 超過極端閾值（例如 10,000），表示系統已超載。
- API Server 直接回傳 `429 Too Many Requests`，並在 response header 中包含 `Retry-After`。
- 結合 per-user rate limiting，避免單一使用者獨佔系統資源。

5. GPU Worker crash 時，如何確保不遺失正在處理的請求？

若 GPU Worker 在呼叫 `batched_sample` 途中 crash，已從 queue 中拉出但未 ACK 的請求需要被重新處理。

如同 Deep Dive 2 中所述，Redis Streams 的 consumer group 原生提供此保證：

- Worker 讀取訊息後，訊息進入該 worker 的 Pending Entries List（PEL）。
- 正常情況下，worker 處理完畢後送出 ACK，訊息從 PEL 中移除。
- 若 worker crash 未 ACK，其他健康的 worker 可在 idle timeout（例如 5 秒）後 claim 這些訊息並重新處理。
- 這等同於 SQS 的 visibility timeout 機制，但完全在 Redis 內部完成，不需要引入外部服務或自行實作。

為什麼重複處理是安全的：

buildnoat.org

- LLM sampling 本身是**無副作用的（side-effect free）**——呼叫 `batched_sample` 不會修改任何外部狀態（不會改資料庫、不會扣款、不會發送訊息）。
- 重複處理的最壞結果是使用者收到一個不同但同樣合理的 completion（因為 sampling 有隨機性），不會造成系統或使用者的任何損害。

6. Redis 同時作為 Request Queue 和 Result Store，如何避免單點故障？

若 Redis 完全不可用，Request Queue 中的待處理請求與 Result Store 中未讀取的結果都會遺失。

首先評估影響嚴重程度：

- 此系統中 Redis 存放的資料是**短暫**、**無狀態**的，request 和 result 的生命週期僅數百毫秒。
- Redis 故障時，API Server 會 timeout 並回傳 504（或 503），使用者 retry 即可恢復。沒有資料庫被修改、沒有錢被扣、沒有不可逆的副作用。

第一層：Redis Cluster + Replica

- 使用 Redis Cluster 進行 sharding，每個 master shard 配置 1-2 個 replica。
- 單個 master 掛掉時，Sentinel 自動將 replica 提升為新 master，failover 時間約 1-2 秒。
- 這解決了大多數生產環境中的故障場景。

第二層：API Server 端的 graceful degradation

- 若 Redis 完全不可用，API Server 直接回傳 `503 Service Unavailable`，並在 header 中包含 `Retry-After`，引導 client 端 backoff retry。

第三層（可選）：分離 Queue 與 Result Store 的 Redis Cluster

- 將 Request Queue 與 Result Store 部署在不同的 Redis Cluster 上，使一方故障不影響另一方。
- 但在多數場景下，單一 Redis Cluster + Replica 已經足夠，過度分離反而增加維運複雜度。

buildmoat.org